An Anti-Reverse Engineering Technique using Native code and Obfuscator-LLVM for Android Applications

Kyeonghwan Lim
Dept. of Computer Science
Dankook University
Yongin 448-701, Korea
10-4657-5663, +82
limkh120@dankook.ac.kr

Jaemin Jeong
Dept. of Software
Dankook University
Yongin 448-701, Korea
31-8005-3239, +82
snorlax@dankook.ac.kr

Seong-je Cho *
Dept. of Computer Science
Dankook University
Yongin 448-701, Korea
31-8005-3239, +82
sjcho@dankook.ac.kr

Jongmoo Choi
Dept. of Software
Dankook University
Yongin 448-701, Korea
31-8005-3242, +82
choijm@dankook.ac.kr

Minkyu Park
Dept. of Computer Engineering
Konkuk University
Chungbuk 380-701, Korea
43-840-3559, +82
minkyup@kku.ac.kr

Sangchul Han *
Dept. of Computer Engineering
Konkuk University
Chungbuk 380-701, Korea
43-840-3605, +82
schan@kku.ac.kr

Seongtae Jhang
Dept. of Computer
Suwon University
Suwon 445-743, Korea
31-220-2126, +82
stjhang@suwon.ac.kr

ABSTRACT
Android applications are exposed to reverse engineering attacks. In particular, the applications written in Java are more prone to reverse engineering in comparison to the applications written in native-code languages such as C or C++ on the Android platform. This is because Java applications are distributed as byte codes, while applications written in native-code languages are distributed as low-level binary codes. In this paper, we propose a new technique to protect Android applications against reverse engineering. Three key characteristics of the proposed approach are as follows. First, we write the main parts of the application in native-code using Android NDK. This not only makes reverse engineering more difficult, but it is also more effective in terms of code reuse. Second, we introduce obfuscation, which hides the intent of the native codes and obscures theirs structure, at the intermediate representation (IR) level. Finally, we integrate an integrity verification scheme which detects whether the critical module of the application has been modified prior to execution of the application. Based on the results of experimentation on five known Android applications, we show that the proposed techniques can be applied without a significant effect on performance.

CCS Concepts
• **Security and privacy →Software and application security;**

Keywords
Reverse engineering; Android application (Android app); Call stack; Integrity verification; Obfuscator-LLVM (O-LLVM)

*Corresponding Author.

RACS '17 , September 20–23, 2017, Krakow, Poland

ACM ISBN 978-1-4503-5027-3/17/09…$15.00
https://doi.org/10.1145/3129676.3129708

1. INTRODUCTION
After collecting and analyzing Android applications (*apps*) for 6 months from Google Play in 2016, McAfee publishes a report uncovering that 37 million pieces of malware were being distributed through the official Android app marketplace [1]. This is because the Android app store is open to the public including individuals or developers in order to upload their apps, thus resulting in a larger number of malware in comparison to Apple's app store. In addition, Android apps consist of byte codes, which are prone to reverse engineering attacks [2-4]. Therefore, there have been frequent cases where malevolent users develop and distribute apps that show malicious behavior including privacy information leak [3, 5].

In order to prevent Android apps from static reverse engineering, several approaches have been proposed. Some employ byte code obfuscation mechanisms such as string obfuscation, renaming, control flow obfuscation and API hiding [6, 7] while others make use of executable file encryption techniques [8, 9]. However, such static prevention techniques can be disarmed by dynamic reverse engineering, which is performed by analyzing the app as it runs.

Prevention techniques for dynamic reverse engineering also have been proposed. These techniques rely on the fact that dynamic reverse engineering requires an emulator or needs to obtain root-level privilege. Root-level privilege is needed to dynamically reverse engineer an app because each Android app operates in its sandbox. Sandbox refers to the territory that is authorized to each app when it is installed, which is only accessible through its unique UID (User Identifier) and GID (Group Identifier). Thus, different apps cannot access or intrude the sandbox of other apps, and analysis such as debugging cannot be performed. With root-level privilege, however, it becomes possible to access the sandbox of another app owned by other UID and GID.

By considering such attacker's behavior, dynamic reverse engineering prevention techniques utilize mechanisms such as Android emulator detection (detecting Android sandboxes), anti-debugging, and root detection (detection of a rooted device on Android) [10-12]. However, attackers also notice these prevention techniques and try to bypass them. For instance, attackers disarm the emulator detection by analyzing the device infrastructure [13],

and are developing elaborate attacks such as anti-debugging through API hooking and code modification [14, 15].

In [19], Lim et al. have proposed a dynamic reverse engineering prevention technique that can defend the elaborate attacks. Specifically, by monitoring the call stack information, the technique detects and responds to not only anti-debugging and rooting but also method (API) hooking. However, the modules developed in [19] were all implemented in Java, thus they are exposed to static reverse engineering techniques. Moreover, they cannot detect code modification attacks.

In this paper, to overcome the limitation of [19], we reinforce the technique into three directions. First, we implement the call stack analysis module which detects method (API) hooking and code modification attacks with native code using Android NDK [16]. Second, as a second layer of defense, we apply obfuscation into the native code using Obfuscator-LLVM (O-LLVM) [17].

Finally, we integrate an integrity verification scheme which detects whether the app has been modified prior to execution. And we integrate the dynamic code loading [18] into our technique so that it can be used for apps that are distributed as executables in the market.

The paper is organized as follows. In Section 2, we survey previous reverse engineering prevention techniques. Details of our proposal is explained in Section 3. Performance evaluation results are discussed in Section 4. Finally, we conclude our work in Section 5.

2. Related Work

In this section, we explore existing anti-reverse engineering techniques for Android apps and compare them with the proposed method.

Code obfuscation techniques [6, 7] have been studied to prevent the reverse engineering of Android apps. Code obfuscation technique convert a program code into a form or structure difficult to analyze with keeping the functionality of an original program. Code obfuscation techniques include identifier mangling, string obfuscation, and control flow obfuscation, etc.

Kovacheva [20] added dead or irrelevant codes to a program to make the program bigger and complex to analyze source codes statically. However, a program with these techniques added are vulnerable to static reverse engineering using dynamic reverse engineering together.

Lim et. al. [19] studied rooting and debugging detection techniques to prevent dynamic reverse engineering. They also protect proposed detection techniques against hooking attacks and code tampering. They add Stub DEX and arrange Stub DEX runs first. Stub DEX defines tasks and corresponding methods to detect rooting and debugging (Table 1).

These methods monitor the runtime call stack and defend attack attempts. However, all techniques were implemented in Stub DEX using Java. They did not explain techniques to protect Stub DEX itself. Therefore, if Stub DEX is reverse-engineered, all proposed techniques can be exposed. They did not consider the possibilities stub DEX can be tampered and repackaged, too.

To overcome the weaknesses above mentioned, we perform integrity checking on Stub DEX, and implement Call Stack-based techniques in the Android native level to make the attacker difficult to reverse engineer them.

Table 1. Checklist and stub DEX's methods

Checklist	Stub DEX's method
Custom image flashing	detectTestKeys( )
Change of system properties	checkForSystemProperties( )
Installed su binary	checkForBinary( )
New command from busybox	checkForBusybox( )
Filesystem attributes	checkForFilesystem( )
Rooting-related apps	detectRootManagementApps( )
Running root process	detectRootProcess( )
Flags for debugging	isDebuggable( )

3. OVERVIEW OF PROTECTION TECHNIQUES

This section proposes an anti-reverse engineering technique using native component and O-LLVM [17] for Android apps. The proposed technique adds two components to Android apps and repackages them. The added components are a Java component and a native component. The Java component is implemented in a DEX file, called Stub DEX. It performs anti-debugging and root detection (*a method to detect if an app is running on a rooted device*), and then loads the original DEX. Since the Stub DEX utilizes dynamic code loading, the proposed technique can be applied to most Android apps in the market whose source codes are not available. The native component is written in C++ and implemented as a shared library. It checks the integrity of Stub DEX and detects method hooking attacks and code modification attacks by examining call stack information. We obfuscate the native component using O-LLVM [17] to prevent reverse engineering attacks. Figure 1 shows the structure of APK before and after applying our technique. The original `classes.dex` file is moved to a resource directory and the Stub DEX is renamed as `classes.dex`. The native component (`libstub.so`) is also included. Then the APK is repackaged and re-signed.

The details of the Stub DEX and `libstub.so` are described in Section 3.1 and Section 3.2, respectively. Section 3.3 explains the difference and improvement over the existing work [19].

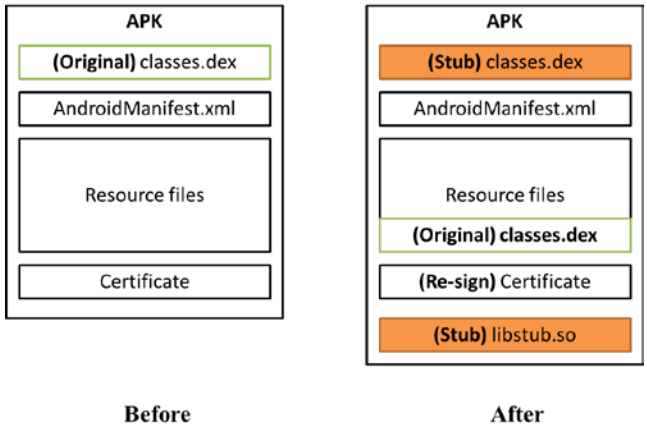

Figure 1. The structure of APK before/after applying our technique

3.1 Stub DEX

The functionality of the Stub DEX is three-fold. First, it invokes `libstub.so` through JNI to check the integrity of the Stub DEX and prevent method hooking and code modification attacks. Second, it performs anti-debugging and root detection to protect the repackaged apps from dynamic reverse engineering. Third, it dynamically loads and executes the original `classes.dex`.

Since our scheme adds the Stub DEX to the original APK without modifying its source codes or `classes.dex`, our technique can be applied to most Android apps without source codes in the market such as Google Play. The value of `android:name` attribute of `Application` element in `AndroidManifest.xml` is modified as the application class of the Stub DEX. When the repackaged app is launched, the Stub DEX is executed first. It invokes `libstub.so` through JNI to check the integrity of the Stub DEX. If the integrity is verified, it performs anti-debugging and root detection to examine whether the app is being reverse engineered. At this time, it invokes `libstub.so` again to examine its call stack to detect and defend method hooking and code modification attacks. The details are explained in Section 3.2. If no attack is detected, the Stub DEX loads and executes the original `classes.dex` by using dynamic code loading technique [19].

3.2 libstub.so

This section describes the role and functions of the native component. The native component, `libstub.so`, provides the functions of integrity checking of the Stub DEX and call stack-based detection of method hooking and code modification attacks.

For integrity checking, our scheme computes the hash value of the Stub DEX using SHA1 and embeds a base64-encoded digest for this hash value in `libstub.so`. When the repackaged app is executed, the digest is compared with the contents of MANIFEST.MF file in META-INF directory. MANIFEST.MF is a file that is generated when an app is signed. It contains the base64-encoded digests for the SHA1 hash value of each file of an app. Hence the integrity is verified if the digest embedded in `libstub.so` is identical to the one for the Stub DEX stored in MANIFEST.MF.

For the call stack-based detection of hooking and code modification attacks, each method listed in Table 1 invokes `libstub.so` to examine the method callstack and identify the sequence of method calls. `libstub.so` obtains the sequence of method calls by calling `getstacktrace()` through JNI. Attackers do not know what method is called by analyzing only the Stub DEX. For example, `detectTestKeys()` in Table 1 is a method in the Stub DEX that detects whether the boot image is a modified one.

Call stack
↓ skaiser.wrapper.TestApplication.onCreate
skaiser.wrapper.TestApplication.doRootCheck
skaiser.wrapper.root_detect.isRooted
skaiser.wrapper. root_detect.detectTestKeys

Figure 2. detectTestKeys method's call stack

When the method is called normally, the sequence of method calls is as shown in Figure 2. In our scheme, the normal sequence of method calls for each method listed in Table 1 is embedded in `libstub.so`. If an attacker tries to hook the method using a framework tool such as Xposed, method calls that are not seen in the embedded normal sequence of method calls are included in the identified one as shown in Figure 3. Our scheme judges that a method is hooked if the identified sequence of method calls is different from the embedded normal one.

Call stack
skaiser.wrapper.TestApplication.onCreate
skaiser.wrapper.TestApplication.doRootCheck
skaiser.wrapper.root_detect.isRooted
skaiser.wrapper. root_detect.detectTestKeys
de.robv.android.xposed.XposedBridge.handleHookedMethod
de.robv.android.xposed.XposedBridge.invokeOriginalMethodNative
skaiser.wrapper. root_detect.detectTestKeys

Figure 3. detectTestKeys method's call stack in hook attack

3.3 Comparison with the existing work

As mentioned in Section 1 and 2, the weak point of the existing work [19] is that the Stub DEX can be easily reverse engineered because it is written in Java. If an attack analyzes the Stub DEX, the anti-reverse engineering mechanisms implemented in the Stub DEX are exposed. The attacker may modify the codes of the Stub DEX and repackage the app to bypass the anti-reverse engineering mechanisms. Or he may hook and modify the method that detects method hooking attacks in order to bypass anti-reverse engineering and/or method hooking detection mechanism.

First, we improve the existing work to protect the Stub DEX from code modification and repackaging attacks. The integrity checking module (inserted in `libstub.so`) verifies the integrity of the Stub DEX to detect code modification as explained in Section 3.2. Second, the call stack-based hooking detection mechanism is implemented in the native component. Since the native component is written in C++ and compiled into binary code, it is difficult to reverse engineer the code. Moreover, our scheme obfuscates the native component using O-LLVM [20] in order to make it more difficult to be reverse engineered. A vulnerable point of our scheme is that an attacker might bypass the proposed techniques by replacing the entire Stub DEX with one that can load the original DEX using dynamic code loading. However, we can overcome this weakness by encrypting the original DEX and decrypting it only through `libstub.so`, which is beyond the scope of this paper.

Table 2 compares this work with the existing work [19]. The objectives of this work is protecting our scheme from reverse engineering as well as detecting dynamic analysis of the original app. The integrity checking module and call stack-based method hooking detection module are written in C++ and implemented as a native component using Android NDK. To protect the native component from reverse engineering, the component is obfuscated using O-LLVM which applies techniques such as Instruction Substitution, Bogus Control Flow, and Control Flow Flattening

Table 2. Comparison with the existing work

	[19]	this work
Implementation	• Java component (Stub DEX)	• Java component (Stub DEX) • native component (libstub.so)
Objectives	• detecting dynamic analysis	• detecting dynamic analysis • call stack-based method

	• call stack-based method hooking detection	hooking detection • detecting modification of the Stub DEX • protecting method hooking detection module from reverse engineering
Source codes	• Java - anti-debugging, rooting detection, method hooking detection (39.4KB) - dynamic code loading (1.7KB)	• Java - anti-debugging, rooting detection (33.5KB) - dynamic code loading (1.7KB) • C++ - integrity checking, method hooking detection (9.1KB)
Improve-ments		• integrity checking of the Stub DEX • native component obfuscation

4. EXPERIMENTS AND EVALUATION

In this section, we have implemented our prototype system with the proposed technique and evaluated the performance of the prototype system. Our prototype has been implemented on Nexus 5 running Android 4.4Kitkat, Linux Kernel 3.4 version. For experiments, we consider a dataset of five real Android apps collected from the Official Android Market. Experiments show that our technique is effective to prevent reverse engineering attacks and code modification attacks.

We applied the proposed technique to the five Android apps and measured execution time overheads of the apps on Nexus 5. At the execution of each app, the time overheads incurred by the proposed technique consist of four components: (1) the time to verify integrity of the Stub DEX (*Integrity verification time*), (2) the time to disable debugging and detect a rooted device (*Anti debugging and Root detection time*), (3) the time to detect method hooking attacks using call stack inspection (*Call stack verification time*), and (4) the time for dynamic loading (*Dynamic loading time*). Each app was executed twenty times and the four components of the time overheads were measured for each execution. The time overheads were calculated as the average of the measured times. The time overheads were shown in Figure 4.

The total time (total overhead) required to the execution of our proposed technique is ranged from 2.45 seconds to 2.88 seconds per app. Such overhead is inevitable for enhancing the security strength of Android apps. In Figure 4, we can see that the largest component of the overheads occurs when the dynamic loading is performed to run the original executable file and the smallest component occurs when the call stack is inspected to investigate hooking behaviors.

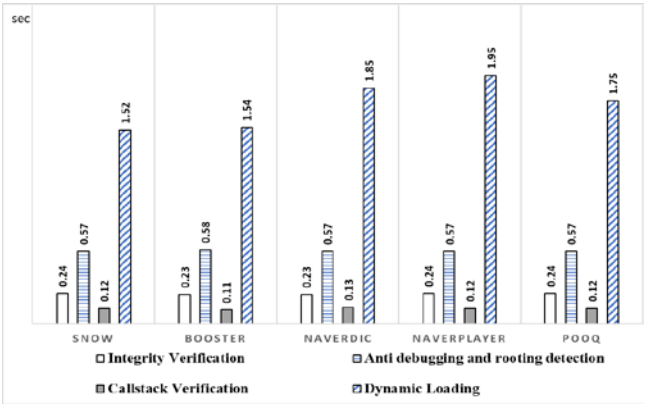

Figure 4. Four parts of the time overheads

In Table 3, we compare the total overheads of the proposed technique with those of the existing work [19]. We can see in Table 3 that the proposed technique incurs higher overhead ranging from 0.22 seconds to 0.33 seconds than the existing work [19]. The reason is that the proposed technique produces additional overhead due to the module for verifying the integrity of the Stub DEX. Other overhead components of the proposed technique are similar to those of the existing work. Table 4 shows every overhead component of the total overheads.

Table 3. Comparison of the total overheads of the proposed technique and the existing work [19]

App name	Original DEX size *(MB)*	Overhead of the existing work [19] (*sec*)	Overhead of the proposed technique (*sec*)
SNOW	8.4	2.22	2.45
BOOSTER	4.8	2.24	2.46
NAVERDIC	5.5	2.45	2.78
NAVERPLAYER	8.1	2.65	2.88
POOQ	7.4	2.46	2.68

5. CONCLUSION

In this paper, we have proposed a new technique using native-code language and O-LLVM [17] in order to protect the main parts of Android apps against reverse engineering attacks. The

Table 4. Detail comparison of the total overheads of the proposed technique and the existing work [19]

	SNOW		BOOSTER		NAVERDIC		NAVERPLAYER		POOQ	
	[19]	The proposed technique	[19]	The proposed technique	[19]	The proposed technique	[19]	The proposed technique	[19]	The proposed technique
Anti-debugging and rooting detection	0.56	0.57	0.58	0.58	0.57	0.57	0.57	0.57	0.57	0.57
Callstack verification	0.13	0.12	0.12	0.11	0.13	0.13	0.12	0.12	0.12	0.12
Dynamic Loading	1.53	1.52	1.54	1.54	1.75	1.85	1.96	1.95	1.77	1.75
Integrity Verification		0.24		0.23		0.23		0.24		0.24
Total overhead	2.22	2.45	2.24	2.46	2.45	2.78	2.65	2.88	2.46	2.68

proposed technique has been developed as two parts in each Android app: a Java component and a native component. We have added the proposed technique to an original Android app and built the original app into a repackage app. The Java component is implemented in a new DEX file, called Stub DEX which is the entry point of the repackage app. The Java component performs

Anti-debugging and root detection, and then loads the original DEX into memory. The native component is written in C++ and implemented as a shared library. It verifies the integrity of the Stub DEX and detects malicious hooking behaviors by inspecting call stack. We have also obfuscated the native component using Obfuscator-LLVM to prevent the shared library from reverse engineering attacks. Finally, we have demonstrated the proposed technique are effective for securing the main parts of Android apps with an acceptable overhead.

6. ACKNOWLEDGMENTS

This research was supported by Basic Science Research Program through the National Research Foundation of Korea(NRF) funded by the Ministry of Science, ICT and Future Planning (No. 2015R1A2A1A15053738).

7. REFERENCES

[1] Snell B. 2016. *Mobile Threat Report What's on the Horizon for 2016*. https://www.mcafee.com/us/resources/reports/rp-mobile-threat-report-2016.pdf

[2] Jang, J., Han, S. Cho, Y., choe, U. and Hong, J. 2014. Survey of Security Threats and Countermeasures on Android Environment. *Journal of Security Engineering*. 11, 1 (Feb. 2014), 01-12. DOI=http://dx.doi.org/10.14257/jse.2014.02.01.

[3] Zhou, W., Zhon, Y. Jiang, X. and Ning, P. 2012. Detecting Repackaged Smartphone Applications in Third-Party Android Marketplaces. In *Proceedings of the 2nd ACM Conference on Data and Application Security and Privacy* (San Antonio, USA, February 07 - 09, 2012). CODASPY'12. ACM, New York, NY, 317-326. DOI=http://dx.doi.org/10.1145/2133601.2133603.

[4] Enck, W., Octeau, D., McDaniel, P. and Chaudhuri, S. 2011. A Study of Android Application Security. In *Proceedings of the 20th USENIX conference on Security* (San Fransicso, USA, August 08 - 12, 2011). SEC '11. USENIX, San Francisco, CA, 21-21.

[5] Chebyshev, V. and Unuchek, R. 2014. *Mobile Malware Evolution: 2013*. https://securelist.com/58335/mobile-malware-evolution-2013.

[6] Piao, Y., Jung, J. and Yi, J. 2013. Structural and Functional Analyses of ProGuard Obfuscation Tool. *The Journal of Korean Institute of Communications and Information Sciences*. 38B, 8 (Aug. 2013). 654-661. DOI=http://dx.doi.org/10.7840/kics.2013.38B.8.654.

[7] Schulz, P. 2012. *Code protection in android*. Lab Report. University of Bonn at North Rhine-Westphalia.

[8] Bangcle. www.bangcle.com. [Online; Accessed on September 10, 2016].

[9] LIAPP. https://liapp.lockincomp.com/. [Online; Accessed on September 10, 2016].

[10] Petsas, T. Voyatzis, G. Athanasopoulos, E. Polychronakis, M. and Ioannidis, S. 2014. Rage against the virtual machine: hindering dynamic analysis of android malware. In *Proceedings of the Seventh European Workshop on System Security* (Amsterdam, Netherlands, April 13 - 16, 2014). EuroSys 2014. ACM, New York, NY, 5. DOI=http://dx.doi.org/10.1145/2592791.2592796.

[11] Cho, H., Lim, J., Kim, H. and Yi, J. 2016. Anti-debugging scheme for protecting mobile apps on android platform. *The Journal of Supercomputing*. 72, 1 (Jan. 2016), 232-246. DOI=http://dx.doi.org/10.1007/s11227-015-1559-9.

[12] Sun, S., Cuadros, A. and Beznosov, K. 2015. Android rooting: Methods, detection, and evasion. In *Proceedings of the 5th Annual ACM CCS Workshop on Security and Privacy in Smartphones and Mobile Devices* (Denver, USA, October 12 - 12, 2015). SPSM '15. ACM, New York, NY, 3-14. DOI=http://dx.doi.org/10.1145/2808117.2808126.

[13] Yang, W., Zhang, Y., Li, J., Shu, J., Li, B., Hu, W. and Gu, D. 2015. AppSpear: Bytecode Decrypting and DEX Reassembling for Packed Android Malware. In *Proceedings of 18th International Symposium* (Kyoto, Japan, November 2 - 4, 2015). RAID 2015. Springer International Publishing, Cham, ZG, 350-381. DOI=http://dx.doi.org/10.1007/978-3-319-26362-5_17.

[14] Costamagna, V. and Bergadano, F. 2016. HOOKDROID: DALVIK DYNAMIC INSTRUMENTATION FOR SECURITY ANALYTICS. *International Journal on Information Technologies and Security*. 8, 1 (2016), 39-52.

[15] Costamagna, V. and Zheng, C. 2016. ARTDroid: A virtual-method hooking framework on android ART runtime. In *Proceedings of the 2016 Innovations in Mobile Privacy and Security* (Egham, SRY, April 06, 2016). IMPS 2016. CEUR-WS, 20-28.

[16] Google. Android NDK. https://developer.android.com/tools/sdk/ndk/index.html.

[17] Welton, R. 2014. Obfuscating Android Applications using O-LLVM and the NDK. http://fuzion24.github.io/android/obfuscation/ndk/llvm/o-llvm/2014/07/27/android-obfuscation-o-llvm-ndk/.

[18] Kim, J., Go, N. and Park, Y. 2015. A Code Concealment Method using Java Reflection and Dynamic Loading in Android. *Journal of the Korea Institute of Information Security and Cryptology*. 25, 1(Feb. 2015), 17-30. DOI=http://dx.doi.org/10.13089/JKIISC.2015.25.1.17.

[19] Lim, K., Jeong, Y., Cho, S., Park, M. and Han, S. 2016. An Android Application Protection Scheme against Dynamic Reverse Engineering Attacks. *Journal of Wireless Mobile Networks, Ubiquitous Computing, and Dependable Applications*. 7, 3(Sep. 2016), 40-52.

[20] Kovacheva, A. 2013. Efficient Code Obfuscation for Android. *In Proceedings of Advances in Information Technology: 6th International Conference* (Bangkok, Thailand, December 12 - 13, 2013). IAIT'13, 104–119. DOI= http://dx.doi.org/10.1007/978-3-319-03783-7_10